AMD ROSS 部署实录与使用心得

> **范围声明**：本文记录的是**部署与验证**，不是"深度使用"。
> 截至 2026-11-01，状态是：MCP server 已装、已注册到两个客户端、已实测能驱动 Vivado 并取回真实工具输出；
> 但**没有用它完成过本项目的任何一次综合或实现**（工程建设走的是 Vitis HLS / Vivado 批处理脚本）。
> 每条结论后面标了证据路径。

一、时间线（全部可核查）

日期	做了什么	证据
2026-10（月内）	参加 AMD 2026 FPGA 赛道线上技术培训，整理成笔记	`amd-fpga-mcp-toolchain/references/training-notes-2026-10.md`（83 行）
2026-10-27 20:31	装入 26 个官方 `agent skill`，全部落到 `~/.agents/skills/`	目录 mtime：`hls-*` (22)、`vivado-*` (6)、`hw-ila/vio-debug` (2)、`vitis-embedded` (1)
2026-10-27 21:09	自建 `amd-fpga-mcp-toolchain` skill（120 行 SKILL + 3 份 reference + 1 个配置脚本）	`~/.agents/skills/amd-fpga-mcp-toolchain/`
2026-10-27 21:11	下载 `vivado-mcp-server.exe`（v2026.9.1，commit 2c4e86e5）	`D:\tools\vivado-mcp-server\`，`--version` 实测
2026-10-27	用自写脚本把 MCP 注册进 ZCode 与 Codex 两个客户端	`~/.zcode/cli/config.json` 的 `mcp.servers.vivado-mcp`；`~/.codex/config.toml` 的 `[mcp_servers.vivado-mcp]`
2026-11-01	**实测**：启动 Vivado 会话 → `version` → `get_parts xck26*` 返回 2 个器件 → 关闭	下方"验证记录"

二、验证记录（2026-11-01，本次实测）

2.1 连接

调用 `vivado_list_sessions` → `No active Vivado sessions. Use vivado_start to create a session.`
说明 server 进程活着、协议通、只是没有会话。

2.2 启动会话

`vivado_start`（TCL 模式、无 GUI）：

```
process_id: 19224
session_id: vivado-20261101-195534-097f5e03
mode: tcl
webserver: http://localhost:8088
```

2.3 取回工具的真实输出

命令一：`version`

```
Vivado v2023.2 (64-bit)
SW Build 4029153 on Fri Oct 13 20:14:34 MDT 2023
```

命令二：`get_parts xck26*`

```
2 xck26-sfvc784-2LV-c xck26-sfvc784-2LVI-i
```

→ 这两个结果都来自本机 Vivado，不是模型记忆里的答案。 这正是 ROSS 那套"两层架构"里 MCP 那一层的作用：
工具是手，模型只负责组织。

2.4 关闭

`vivado_stop` → `Vivado closed`。进程干净退出。

三、踩到的真实问题（这是本文最有价值的部分）

3.1 `VIVADO_PATH` 指到了错误的安装

自写配置脚本 `configure_amd_toolchain.py` 的路径探测逻辑是：

```
cands  = sorted(glob.glob(r"D:/Vivado/Vivado/*/bin/vivado.bat"), reverse=True)
cands += sorted(glob.glob(r"C:/Xilinx/Vivado/*/bin/vivado.bat"), reverse=True)
cands += sorted(glob.glob(r"D:/*/Vivado/bin/vivado.bat"), reverse=True)   # ← 这里
```

`--check` 在本机的输出：

```
Vivado 版本:
    D:\Vivado\Vivado\2023.2\bin\vivado.bat
    D:\2026.1\Vivado\bin\vivado.bat
选用     : D:\Vivado\Vivado\2023.2\bin\vivado.bat
```

两个问题：

1. **选中的是 2023.2**，而本项目 10-30 起已整链迁移到 2026.1。
2. **它找到的那个 2026.1 也是废的** —— `D:\2026.1\Vivado\bin` 是安装器的外层目录，
   缺 `zynq` / `zynquplus` 器件数据（`get_parts xck26*` 零结果）；
   真正能用的是 `D:\2026.1\2026.1\Vivado\bin\vivado.bat`（多一层）。

结果：MCP 现在是能用的，但它驱动的是 2023.2，和我们提交的工程版本不一致。

修法（一行）：把注册项里的 `VIVADO_PATH` 改成

```
D:\2026.1\2026.1\Vivado\bin\vivado.bat
```

并重跑 `python scripts/configure_amd_toolchain.py --vivado "D:\2026.1\2026.1\Vivado\bin\vivado.bat"`。
改完要重启客户端才生效（MCP 配置在启动时读取）。

3.2 官方 skill 的"自述边界"写得很好，值得学

`amd-fpga-mcp-toolchain/SKILL.md` 开头有一段：

> It was not executed or verified on this machine while writing it.
> Anything marked (`lecture`) came from the session; anything marked (`verified here`) was actually measured on this workstation.

这是**给 agent 看的免责声明**：告诉它哪些内容可信、哪些要先验证。这个写法本身就是一条 skill 设计经验。

3.3 培训材料与自己的工程不在一个赛道

`training-notes-2026-10.md` 里有几条是**智能体赛道**的规则（不需要板卡照片、验证窗口录屏可替代现场演示、无基线任务），
用到自主选题赛道上会误导。整理笔记时如果不在条目上标赛道，后面很容易用错。

四、使用心得（真做了什么就说什么）

4.1 "两层"这个划分是成立的

- **MCP = 手**：跑脚本、控制流程、执行标准化 API。好处是**答案来自本地工具**，模型无法编造。
  实测那次 `get_parts xck26*` 返回 2 个具体型号，这种输出模型猜不出来。
- **Skill = 脑**：把"先做什么、检查什么、什么时候停"固化成文档。

这两件事分开之后，责任也分开了：工具错了是环境问题，流程错了是 skill 写得不好，模型跑偏是提示词的问题。定位变清楚。

4.2 skill 应该是"链"而不是"一篇大文档"

培训里给的这条建议（`training-notes-2026-10.md` 第 61–66 行）在实操中确实成立：

> Prefer a skill chain: split a complex task into narrow skills and chain them, rather than one skill that must survive a long context.
> Fix the workflow so the agent cannot skip verification, and make it stop at decision points.

对照官方那 26 个 skill 就能看出这个结构：`hls-architect` 是入口，它自己去调 `hls-dataflow`、`hls-pipeline`、`hls-perf-pragma`；
`hls-optimize` 是入口，调 `hls-ii-diagnose`、`hls-synth-report`。
单个 skill 只负责一件事，职责清楚。

4.3 "停下来问"比"一次做完"更值钱

培训里那个时序例子最能说明问题：

> timing methodology check found 18 critical warnings from an undefined clock B, stopped to ask for its frequency, received 150 MHz, then proposed a 1 ns IO delay which the engineer accepted.

它在**缺信息的时候停下问**，而不是猜一个频率往下做。这一点和我们自己项目里的经验完全一致——本项目 13 个坑，凡是"猜着往下做"的都返工了，凡是"设计一个实验把假设分开"的都一次定住。

4.4 部署成本比想象的低，但"跑通"和"有用"之间还有距离

- 装 skill 是一分钟的事（复制目录）。
- 装 MCP server + 注册两个客户端，有脚本的话十分钟。
- 但**真正用它干活**需要满足：工具版本与工程一致、代码库能被它读到、验收判据能被它执行。

我们只走到了第三步的前半段（能驱动工具），没走到"用它完成一次综合"。

五、给 AMD 的改善建议（按价值排序）

1. **MCP 注册应该校验"这个 Vivado 能用"**，而不只是"这个路径存在"。
   建议注册时跑一次 `get_parts xck26*`（或目标器件族），空结果就不要写进配置，并给出提示。
   我们这次就是"路径存在但缺器件数据"，如果不手动 `--check` 根本发现不了。
2. **路径探测不要用单层通配**。统一安装器的布局是 `<install>/<version>/Vivado/bin`（两层），
   但很多人机器上还有 `D:/Vivado/Vivado/<ver>/bin` 这种老布局。两者混在一起排序，老版本会抢到。
3. `VIVADO_PATH` **与实际工程版本的一致性没有检查**。工程用 2026.1、工具链用 2023.2 时，
   skill 不会报警，只有到 `set_part` 报错才发现。

4. **培训材料请按赛道分册**。智能体赛道的规则（无基线、可录屏代替现场演示）与自主选题赛道差别很大，
   混在一份笔记里容易误用。
5. `get_parts` **这类查询建议在 MCP 层做缓存**。每次开会话都全量枚举器件族，在慢盘上很贵。

六、我们自己的 skill（这才是本项目真正沉淀的东西）

官方 26 个 skill 解决的是"怎么用 AMD 工具"，解决不了"怎么证明你的加速器算对了"。
我们自己的四个 skill 针对后者：

skill	解决什么	来源
`fpga-accel-verify`	定点加速器从 csim → cosim → 板级的三层对拍纪律；"结果偏一个常数""参数写不进""实测比估计慢"这类只有上板才暴露的问题怎么定位	本项目的 13 个坑
`fpga-board-bringup`	会挂板子的操作清单、如何设计一次分开多个假设的实验	本项目的板级排障
`amd-fpga-mcp-toolchain`	ROSS 的部署流程（含上面那个版本一致性坑）	本次部署
`local-verilog-model`	离线 Verilog 生成与判分	独立项目

`fpga-accel-verify` **的设计原则**（如果评委问"你们的 skill 和官方有什么不同"）：

- 官方 skill 面向"设计阶段"（怎么写 HLS、怎么收敛时序）；
  我们的面向"**验收阶段**"（怎么证明板上算的和模型一致）。
- 它强制一个顺序：先 csim、再 cosim、再上板，每层都要有参考值；**不允许跳层**。
- 它把"误差的符号"当一等公民：对安全相关系统，偏保守还是偏乐观，比误差大小更重要。
  本项目最终能写出"板子从不比浮点参考更乐观超过 9.8 mm"，就是这条纪律的产物。

七、这份材料在比赛里怎么用

对应"大模型协作与技能包"那一项，可以写四句话：

1. **我们部署了 AMD 官方的 agent 工具链**（26 个 skill + Vivado MCP server 2026.9.1），
   并实测驱动 Vivado 取回真实器件列表（`get_parts xck26*` → 2 个型号）。
2. **我们发现了它的一个部署缺陷**：路径探测会把"存在但器件数据不全"的安装选中，
   并给出了修法（注册时校验器件族、路径探测按安装布局分层）。
3. **我们自己沉淀了验收侧的 skill**（`fpga-accel-verify`），补官方 skill 的空档。

4. **我们对两者的边界是清楚的：**工具链只用到"能驱动"这一步，工程实现仍走批处理脚本，没有夸大使用深度。

> 官方培训原话说 skill 包是加分项、"发现官方工具的问题"也加分（`training-notes-2026-10.md` 第 68–74 行）。
> 上面第 1、2 两条正好对应这两项，而且是可复现的。

附：证据文件索引

内容	路径
培训笔记（83 行，标注 lecture/verified）	`~/.agents/skills/amd-fpga-mcp-toolchain/references/training-notes-2026-10.md`
部署 skill 正文	同上 `SKILL.md`（120 行）
客户端配置指南	同上 `references/client-setup.md`
时序调试参考	同上 `references/timing-debug.md`
注册脚本（154 行，幂等、自动备份）	同上 `scripts/configure_amd_toolchain.py`
ZCode 侧注册项	`~/.zcode/cli/config.json` → `mcp.servers.vivado-mcp`
Codex 侧注册项	`~/.codex/config.toml` → `[mcp_servers.vivado-mcp]`
MCP 二进制与版本	`D:\tools\vivado-mcp-server\vivado-mcp-server.exe`（v2026.9.1）
官方 26 个 skill	`~/.agents/skills/{hls-*,vivado-*,hw-*-debug,vitis-embedded}`
自研验收 skill	`~/.agents/skills/fpga-accel-verify/SKILL.md`